

Überblick + Grundlagen

Was ist WPF?

- Windows Presentation Foundation
 - Deklarative Beschreibung von User Interfaces
 - Bibliothek von Klassen für Oberflächengestaltung
 - nutzt DirectX anstelle von GDI/GDI+
 - trennt UI und Präsentationslogik
 - XAML als XML-basierte Oberflächensprache
-
- seit .NET Framework 3.0 (2006) verfügbar
 - seit .NET Core 3.0 (2019) vollumfänglich in Core verfügbar

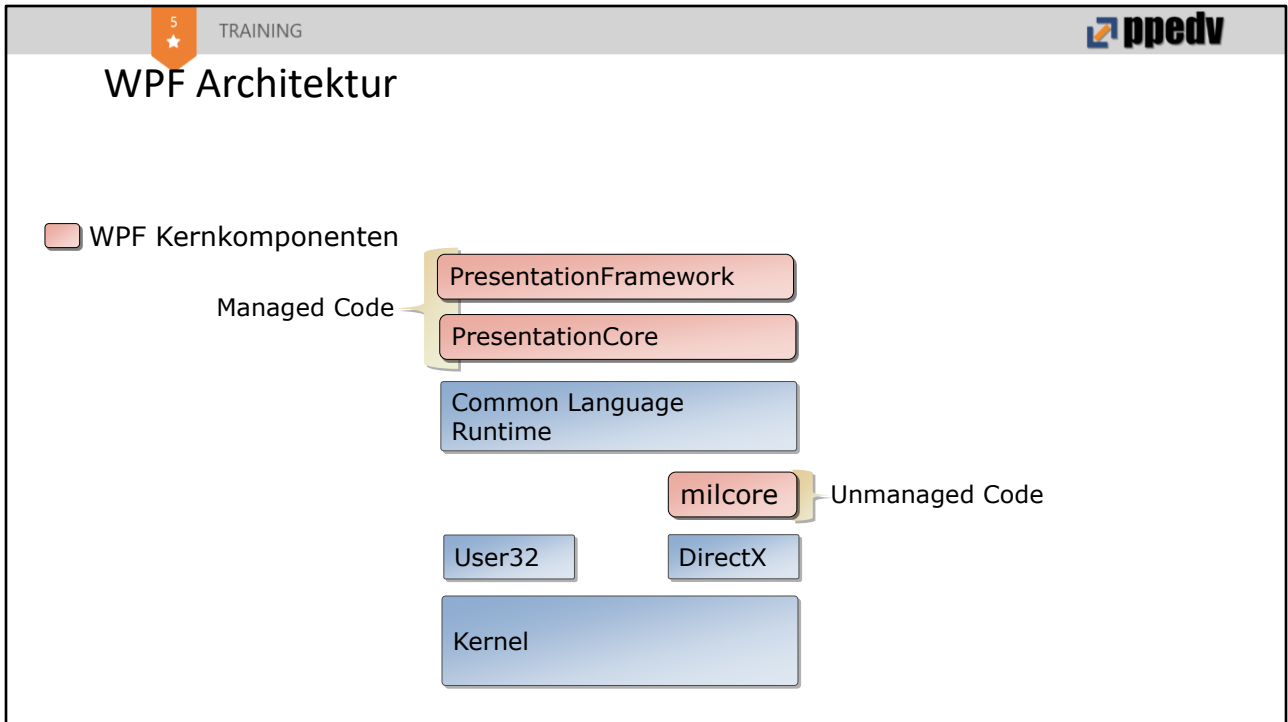
2

Windows Presentation Foundation (WPF) ist eine Technologie von Microsoft, die für die deklarative Beschreibung von Benutzeroberflächen (User Interfaces) verwendet wird. Es handelt sich um eine Bibliothek von Klassen, die für die Gestaltung von Oberflächen in Windows-Anwendungen entwickelt wurde.

Eine der wichtigsten Merkmale von WPF ist, dass es die Windows-eigene DirectX- anstelle von GDI/GDI+-Schnittstelle für die Darstellung von Grafiken und die Benutzeroberfläche verwendet. Dadurch bietet WPF eine hardwarebeschleunigte Grafikdarstellung und ermöglicht ansprechende visuelle Effekte, was allerdings dazu führt, dass WPF nativ ausschließlich auf Windowsmaschinen läuft.

Die Benutzeroberfläche wird in XAML (eXtensible Application Markup Language), einer XML-basierten Sprache, deklarativ beschrieben, während die Präsentationslogik in der zugrunde liegenden Programmiersprache (z. B. C#) implementiert wird. Dadurch kann, insbesondere unter Verwendung von Entwurfsmustern wie MVVM (Model-View-ViewModel) eine Trennung zwischen UI und Präsentationslogik erreicht werden.

WPF wurde erstmals mit dem .NET Framework 3.0 im Jahr 2006 eingeführt und hat sich seitdem als eine der Haupttechnologien für die Entwicklung von Windows-Anwendungen etabliert. Mit der Einführung von .NET Core 3.0 im Jahr 2019 ist WPF nun auch vollständig in der .NET Core-Plattform sowie den aktuellen Nachfolgern (.NET 6) verfügbar.



WPF besteht aus drei Hauptkomponenten: PresentationFramework, PresentationCore und milcore.

PresentationFramework ist eine verwaltete (managed) Komponente von WPF, die für die Erstellung und Verwaltung der Benutzeroberfläche zuständig ist. Sie bietet verschiedene Steuerelemente, Layout-Manager und visuelle Effekte, um interaktive und ansprechende Benutzeroberflächen zu erstellen. Dies ist die Komponente, mit der der Benutzer direkt interagiert.

PresentationCore ist eine weitere verwaltete Komponente, die sich um die Darstellung und Interaktion mit den visuellen Elementen kümmert. Sie stellt Funktionen für die Grafikdarstellung, das Rendern von Elementen und die Ereignisverwaltung bereit.

Die dritte Komponente, milcore (Media Integration Layer Core), ist tatsächlich kein Bestandteil von WPF, sondern ein Teil der unterliegenden Infrastruktur. Milcore ist ein hoch optimierter unverwalteter (unmanaged) Code, der direkt mit der DirectX API kommuniziert. Es handelt sich um eine Schlüsselkomponente, die die Grafikdarstellung und Leistung in WPF ermöglicht, indem sie den Zugriff auf die DirectX-Funktionalität erleichtert.

Zusammen arbeiten diese drei Komponenten eng zusammen, um die Funktionalität und Leistung von WPF bereitzustellen.

Eigenschaften von WPF

- Performantes Rendering der GUI dank DirectX-Hardwarebeschleunigung
- Code-Portierbarkeit auf andere XAML-Plattformen (UWP, MAUI)
- dynamische Anpassung der GUI auf verschiedene Bildschirmgrößen
- deklarative Programmierung der GUI sorgt für weniger C#-Quellcode

WPF bietet dank der DirectX-Hardwarebeschleunigung ein performantes Rendering der grafischen Benutzeroberfläche. Durch die direkte Kommunikation mit DirectX kann WPF die Ressourcen der Grafikkarte optimal nutzen, um die Darstellung von UI-Elementen zu beschleunigen. Dies führt zu einer flüssigen und ansprechenden Benutzererfahrung, insbesondere bei komplexen und grafikintensiven Anwendungen.

Da WPF auf der XAML-Syntax basiert, kann der XAML-Code relativ einfach zwischen XAML-Plattformen wie Universal Windows Platform (UWP) und Xamarin geteilt und wiederverwendet werden. Dies ermöglicht eine effiziente Entwicklung von plattformübergreifenden Anwendungen und reduziert den Aufwand für die Anpassung des Codes an verschiedene Zielplattformen.

WPF bietet auch Mechanismen zur dynamischen Anpassung der Benutzeroberfläche auf verschiedene Bildschirmgrößen. Dank der flexiblen Layout-Manager und der Unterstützung für responsive Designs kann die GUI automatisch auf unterschiedliche Bildschirmgrößen und -auflösungen reagieren. Dies erleichtert die Entwicklung von Anwendungen, die auf verschiedenen Geräten und Bildschirmgrößen gut funktionieren, ohne dass der Entwickler spezifische Anpassungen vornehmen muss.

Da die Benutzeroberfläche in XAML deklarativ beschrieben wird, kann der Entwickler visuelle Layouts und Steuerelemente einfach definieren, ohne umfangreichen C#-Code schreiben zu müssen. Dies fördert die Trennung von UI und Präsentationslogik und erleichtert Wartung und Lesbarkeit des Codes.

Umgebungen

- Entwicklung:
 - Visual Studio 2008 – 2022
- Design:
 - Blend
 - GUI-Design (insb. Animationen)
 - Expression Design
 - Grafiken (in XAML)

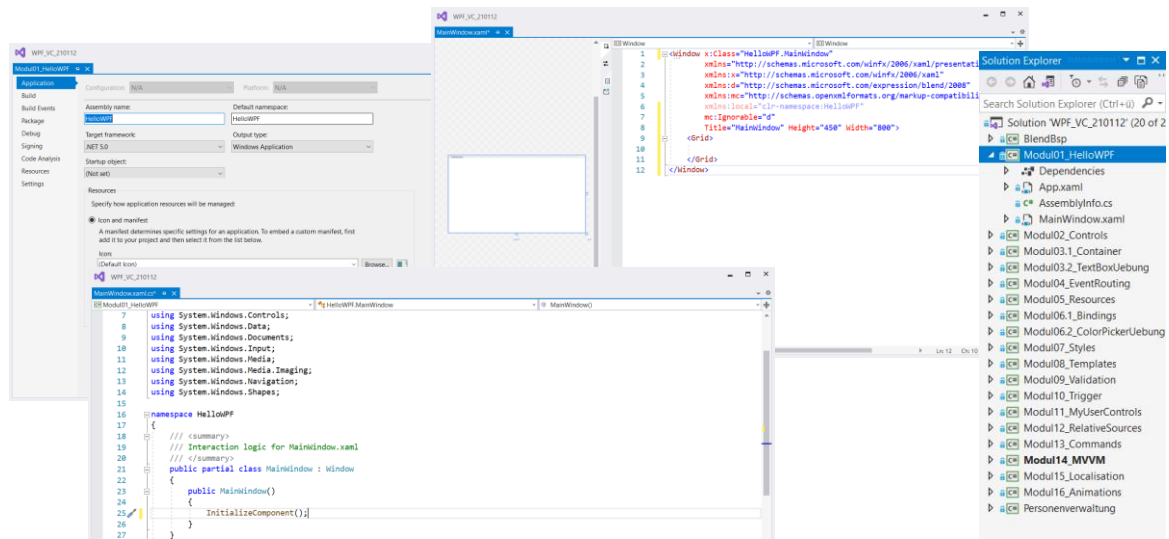


Zur Entwicklung einer WPF-App wird Visual Studio empfohlen. Dies ist ab der Version 2008 möglich, wobei je nach verwendeter .NET-Version die aktuelleren VS-Versionen vorzuziehen sind.

Zum Design der Anwendung kann der zusammen mit Visual Studio ausgelieferte BLEND-Designer verwendet werden, wobei die aktuelleren Versionen von Visual Studio nahezu alle Funktionen von BLEND übernommen haben, was letzteren bis auf wenige Fälle (z.B. der GUI-unterstützten Programmierung zeitgesteuerter Animationen) obsolet macht.

Zu Erstellung von Vektor-Grafiken mittels XAML bietet sich Microsofts Designsoftware ExpressionDesign an.

Visual Studio Projekt



Zur Erstellung eines neuen WPF-Projekts unter Visual Studio 2022 wähle als Projekt-Template ‚WPF-App (.NET)‘ aus. Visual Studio wird automatisch zwei *.xaml-Dateien erstellen:

- MainWindow.xaml ist ein Platzhalter-Datei eines ersten Fensters, welche direkt verwendet werden kann.
- App.xaml repräsentiert das App-Objekt der Applikation und wichtige globale Eigenschaften, wie z.B. die StartupUri, über welche man das Startfenster definieren kann.

XAML

- Extensible Application Markup Language
- XML-basierte Sprache
- XAML als „neue“ Overflächendesignsprache
 - WPF / UWP / MAUI
 - Ursprünglich auch für Silverlight / Windows Phone
- Vektororientiert
 - Skaliert besser
 - Hardwarebeschleunigung
- Wird vor dem eigentlichen Kompilieren durch einen XML-Parser in C#-Code interpretiert



Die Extensible Application Markup Language (XAML) ist eine XML-basierte Sprache, die als Oberflächendesignsprache in verschiedenen Microsoft-Plattformen wie WPF, UWP und MAUI verwendet wird. Es ermöglicht die deklarative Beschreibung von Benutzeroberflächen und stellt eine intuitive und strukturierte Art und Weise bereit, um UI-Elemente zu erstellen und ihre Eigenschaften festzulegen.

Im Gegensatz zu reinem C#-Code bietet XAML eine vektororientierte Beschreibung von UI-Elementen. Dies bedeutet, dass die grafischen Elemente in skalierbarer Form beschrieben werden, sodass sie sich an verschiedene Bildschirmgrößen anpassen und eine hohe Qualität beibehalten können.

Bevor XAML in die eigentliche Anwendung kompiliert wird, wird es durch einen XML-Parser in C#-Code interpretiert. Dieser Parser läuft von oben nach unten durch den XAML-Code, wodurch bei Syntaxfehlern häufig im darunterstehenden Code Folgefehler gemeldet werden. Auch wenn XAML interpretiert wird, müssen alle verwendeten Klassen und Objekte in kompilierter Form vorliegen.

XAML VS. Code

```
<Window ...>
  <StackPanel>
    <TextBox Margin="20" Text="Inhalt"/>
    <Button Margin="10" HorizontalAlignment="Right" Content="OK"/>
  </StackPanel>
</Window >
```

```
public MainWindow()
{
    // StackPanel
    StackPanel sp = new StackPanel();
    this.Content = sp;

    // TextBox
    TextBox tb = new TextBox();
    tb.Margin = new Thickness(20);
    tb.Text = "Inhalt";
    sp.Children.Add(tb);

    // Button
    Button btn = new Button();
    btn.Margin = new Thickness(10);
    btn.Content = "OK";
    btn.HorizontalAlignment =
        HorizontalAlignment.Right;
    sp.Children.Add(btn);
}
```

Im obigen Beispiel ist ein Vergleich zwischen XAML-Code und einen C#-Code zu sehen. Beide Codes bewirken dasselbe.

Zu beachten ist insbesondere:

1. Die Öffnung eines XAML-Tags (<>-Klammern) instanziiert immer ein neues Objekt dieser Klasse
2. Die Position des Tags gibt an, welcher Eigenschaft des übergeordneten Objekts dieses neue Objekt zugeordnet wird (StackPanel in Window.Content, TextBox in StackPanel.Children)
3. Auch Eigenschaften, welche komplexe Objekte benötigen (z.B. Margin) können in den Tags als String-Attribute geschrieben werden. Der Parser instanziiert diese Objekte dann automatisch

Alles in allem kann alles, was in XAML ausgedrückt werden kann, auch in C# geschrieben werden, wobei der XAML-Code in vielen Fällen weniger Zeichen benötigt.

Regeln

- Basierend auf XML
 - Case-Sensitiv
 - Nur ein Root-Element `<Window x:Class="HallowPF.MainWindow" [...] > ... </Window>`
 - UI-Elemente per Tags deklarieren:
 - <Einleitung> </Ausleitung> `<Button>Klick Mich</Button>`
 - <Einzeln/> `<Button Content="Klick Mich"/>`
 - Kommentar `<!--Mein Kommentar-->`
 - Namensräume `xmlns:local="clr-namespace:HallowPF,,`
 - Attribute

XAML basiert auf den gleichen Syntanx-Regelen wie XML. Das heißt

1. XAML unterscheidet in allen Fällen zwischen Groß- und Kleinbuchstaben
2. Jedes XAML-Dokument darf nur ein Root-Element beinhalten, d.h. sie beginnt und endet mit Tags von diesem Objekt. Wenn die Datei einen Code-Behind hat (gleicher Dateiname mit *.xaml.cs-Endung) muss die darin definierte Klasse von der gleichen Klasse erben, welche als Root-Element definiert ist.
3. Die Element innerhalb des Dokuments werden in Form von Tags definiert, welche immer mit der Klassenbezeichnung des Objekts beginnen. Dies kann entweder durch Start- und Endtag geschehen, wobei Eigenschaftsattribute im Starttag festgelegt werden und der Endtag mit einem / beginnt, oder in Form eines Einzeltags, welche die Attribute beinhaltet und mit einem / endet. Ein Tag muss immer erst beendet werden(durch >) bevor der nächste Tag beginnen kann.
4. Kommentare werden in Form spezieller Tags angelegt (<!-- Mein Kommentar -->)
5. Jeder verwendete Namensraum muss im Starttag des Root-Elements durch ein xmlns-Attribut und ein Präfix definiert werden (siehe folgende Seite)
6. Eigenschaften werden entweder in Form von Attributen im Starttag oder durch ‚Auslagern‘ der Eigenschaft gesetzt (siehe übernächste Seite)

Namensräume

- Kern-Namensräume
 - <http://schemas.microsoft.com/winfx/2006/xaml/presentation>
 - Enthält wichtige Steuerelemente (UserControl, Grid, Button, ...)
 - Ohne Präfix
 - <http://schemas.microsoft.com/winfx/2006/xaml>
 - XAML-Namensraum
 - XAML-Funktionen (z.B. x:Class, x:Name, x:Key) -> Compilerbefehle
- Achtung: Keine „Webseite“, sondern nur ein eindeutiger Name !
- Weitere Namespaces werden im Root-Element mittels `xmlns:??` definiert, wobei die ?? ein frei wählbares Präfix repräsentieren
- Der Zugriff auf Objekte eines Namensraums erfolgt über das Präfix

In XAML verwendete Namenräume (Namespaces) müssen im Root-Element des XAML-Dokuments mittels eines "xmlns"-Attributs definiert werden.

Standardmäßig werden die Kern-Namensräume "<http://schemas.microsoft.com/winfx/2006/xaml/presentation>" für wichtige Steuerelemente und "<http://schemas.microsoft.com/winfx/2006/xaml>" für XAML und Compiler-spezifische Funktionen automatisch eingefügt. Diese Namensräume dienen als eindeutige Bezeichnungen und sind trotz ihres Musters keine Webseiten-URLs.

Zusätzliche Namensräume können mit dem "xmlns"-Attribut und einem frei wählbaren Präfix, über welche auf diese zugegriffen wird, eingefügt werden, um die spezifische Elemente und Funktionen (z.B. UserControls, Converter-Objekte, Validierungsklassen, Modelobjekte) zuzugreifen.

Eigenschaften festlegen

- Attribut Syntax

```
<TextBox Text="Hallo Welt"></TextBox>
```

- Property-Element-Syntax („Auslagern“)

```
<TextBox>
  <TextBox.Text>
    Hallo Welt
  </TextBox.Text>
</TextBox>
```

- Content-Property (Default)

```
<TextBox>Hallo Welt</TextBox>
```

```
<TextBox>
  <TextBox.Background>
    <LinearGradientBrush>
      <GradientStop Offset="0"
        Color="White"></GradientStop>
      <GradientStop Offset="1"
        Color="Black"></GradientStop>
    </LinearGradientBrush>
  </TextBox.Background>
</TextBox>
```

In XAML können Eigenschaften von Objekten auf verschiedene Weise gesetzt werden:

1. **Attribut-Syntax:** Die meisten Eigenschaften können direkt als Attribute innerhalb des entsprechenden Tags in der XAML-Datei angegeben werden. Dafür ist es notwendig, dass der XML-Parser die den Datentyp der Eigenschaft aus dem XAML-String extrapolieren kann. Bei den meisten Basis- sowie den meisten WPF-eigenen Datentypen ist dies möglich.
2. **Property-Element-Syntax („Auslagern“):** Werden Start- und Endtag für ein Objekt verwendet können dazwischen mittels weiterer Start- und Endtags Eigenschaften nach dem Schema `<Objekt.Eigenschaft>Wert</Text>` gesetzt werden. Dies ermöglicht die Belegung von Eigenschaft mit komplexen sowie abgeleiteten Objekten und unabhängig von den Umwandlungsfähigkeiten des Parsers.
3. **Default-Eigenschaft:** Die meisten UI-Elemente haben eine Default-Eigenschaft, die automatisch verwendet wird, wenn keine explizite Eigenschaftsangabe gemacht wird und der Wert zwischen Start- und Endtag gelegt wird. Bei Layoutcontainern ist die die Children-Eigenschaft. Bei anderen Elementen ist die im Normalfall die Content-Eigenschaft (wenn vorhanden) oder eine andere Eigenschaft, welche der Beschriftung oder ähnlichem dient (z.B. Text bei der TextBox).

Typenkonversion

- Height und Width
 - Properties sind Nummern und XAML liefert diese als String
- Bsp. Komplexe Eigenschaften
 - HorizontalAlignment/ VerticalAlignment
 - Margin
 - einzelner Wert (alle 4 Seiten)
 - zwei Werte (left & right und top & bottom)
 - vier Werte (left, top, right, bottom)
- Intern über StringConverter/ TypeConverter realisiert

Das XAML-Parsersystem verwenden Typkonvertierungen, um Werte in den entsprechenden Datentypen umzuwandeln.

Bei Eigenschaften wie z.B. Height und Width werden die Werte als Zeichenketten angegeben und implizit in numerische Datentypen, wie Double, konvertiert.

Komplexe Eigenschaften wie HorizontalAlignment und VerticalAlignment werden als Zeichenketten angegeben und durch den XAML-Parser mithilfe eines StringConverters in Enum-Formate umgewandelt.

Die Margin-Eigenschaft, die den Abstand eines Elements definiert, kann als einzelner Wert, zwei Werte oder vier Werte angegeben werden. Der XAML-Parser verwendet einen TypeConverter, um die Zeichenketten in ein Thickness-Objekt mit den entsprechenden Werten zu konvertieren.

Diese Konvertierungen erfolgen intern und sind Teil des XAML-Parsingprozesses. Sie ermöglichen es, die Werte korrekt zu interpretieren und die entsprechenden Objekte zur Laufzeit zu erstellen.

x:Name

- Die Eigenschaft x:Name definiert den Bezeichner eines UI-Elements
- muss eindeutig sein
- Ermöglicht C# seitigen Zugriff auf das Element
- Auch wichtig für weitere Referenzierungen (z.B. in Bindung, etc)

```
<Button x:Name="Btn_Ok" Content="OK"/>
```

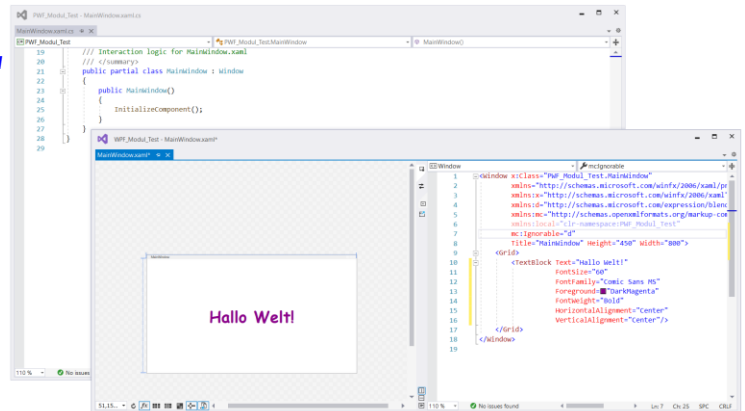
```
Btn_Ok.Background = new SolidColorBrush(Colors.Green);
```

Die "x:Name"-Eigenschaft ist eine spezielle Eigenschaft, die Teil des XAML-Namespaces ist. Sie wird verwendet, um einem Element einen eindeutigen Namen zuzuweisen, der im Code-Behind oder in XAML-Bindungen referenziert werden kann. Der "x:Name"-Wert wird zur Laufzeit in einer Variable des entsprechenden Typs im Code-Behind generiert und kann verwendet werden, um auf das Element zuzugreifen und damit zu interagieren.

Die alternative Name-Eigenschaft (ohne „x:“) tut weitestgehend das gleiche, bietet aber unter Umständen nicht den selben vollen Funktionsumfang wie x:Name, weshalb x:Name die bevorzugte Variante ist.

XAML <-> CodeBehind

- Window.xaml <-> Window.xaml.cs / Window.g.i.cs
- *.xaml:
 - `x:Class="HallowPF.MainWindow"`
- *.cs:
 - Klassendefinition
 - EventHandler
 - Konstuktur
 - InitializeComponent()
 - > *.g.i.cs



14

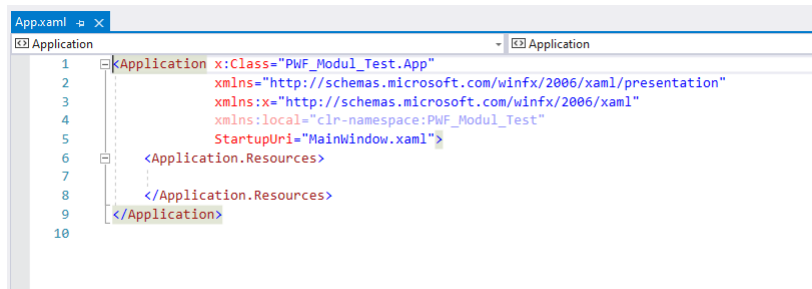
In WPF werden XAML-Dateien (.xaml) mit den Code-Behind-Dateien (.xaml.cs) und der generierten Datei (*.g.i.cs) verknüpft. Die XAML-Datei enthält die Oberflächenbeschreibung, die Code-Behind-Datei den C#-Code der Klasse inkl. Konstruktor und die generierte Datei den automatisch generierten Code, in welchem hauptsächlich der Initialisierungsbefehl für die XAML-Oberfläche durch die Funktion InitializeComponent() abgelegt ist.

Die Verknüpfung der beiden C#-Dateien erfolgt durch eine partielle Klasse. Die Klassennamen in XAML- und Code-Behind-Dateien müssen dieselben sein und das Root-Element der XAML-Datei muss mit der Mutterklasse in den C#-Dateien übereinstimmen. So kann der Code in der Code-Behind-Datei auf die UI-Elemente in der XAML-Datei zugreifen.

Das x:Class-Attribut in der XAML-Datei identifiziert die zugehörige Code-Behind-Klasse. Der XAML-Parser nutzt dies, um die Verbindung herzustellen.

Diese Mechanismen ermöglichen die korrekte Funktionalität zwischen UI und Code. Durch Ereignisse, Eigenschaften und Methoden in der Code-Behind-Datei kann die Benutzeroberfläche manipuliert werden.

App.xaml



```
1 <Application x:Class="PWF_Modul_Test.App"
2   xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
3   xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
4   xmlns:local="clr-namespace:PWF_Modul_Test"
5   StartupUri="MainWindow.xaml">
6   <Application.Resources>
7   </Application.Resources>
8 </Application>
```

- StartupUri
- applikationsweite Ressourcen
- App.xaml.cs

15

Den Dateien "App.xaml" und "App.xaml.cs" haben wichtige Funktionen innerhalb einer WPF-App. "App.xaml" ist eine XAML-Datei, die Anwendungseinstellungen und globale App-Ressourcen definiert. Sie dient auch als Einstiegspunkt der Anwendung. "App.xaml.cs" ist die Code-Behind-Datei von "App.xaml" und enthält den C#-Code für die Anwendungslogik.

In "App.xaml" können Ressourcen wie Styles und Vorlagen definiert werden, die von allen Fenstern und Benutzeroberflächen in der Anwendung gemeinsam genutzt werden. Die Datei enthält auch die Startup-Eigenschaft, die angibt, welches Fenster zuerst angezeigt werden soll.

"App.xaml.cs" erbt von der Klasse "Application" und enthält den Code, der den Anwendungslebenszyklus steuert. Hier können Eventhandler für Anwendungsereignisse wie das Starten, Beenden, Aktivieren oder Deaktivieren der Anwendung definiert werden. Der Code kann angepasst werden, um benutzerdefinierte Initialisierungen, Konfigurationen oder Ressourcenverwaltungen durchzuführen.